



Modelado UML

Sistema Restaurante

Aprendiz:

Sergio Andrés García Salazar C.C. 1076907008

Félix Antonio Murcia Cedeño C.C. 1137624311

Diego Andrés Ávila C.C. 1077724018

Instructor:

Carlos Julio Cadena

Ing. Sistemas

Servicio Nacional De Aprendizaje

Centro De La Industria, La Empresa Y Los Servicios.

Análisis Y Desarrollo De Software. (3239188)

Neiva

2026

Contenido

1	Introducción	1
1.1	Propósito	1
1.2	Contexto del sistema	1
1.3	Diagramas incluidos.....	2
1.4	Convenciones y códigos usados.....	3
2	Diagrama de casos de uso	4
2.1	Actores	5
2.2	Catálogo de casos de uso	6
2.3	Relaciones <<include>>	7
2.4	Especificación de los casos más importantes.....	8
2.4.1	CU-04 Registrar pedido / comanda.....	8
2.4.2	CU-05 Anular ítem	8
2.4.3	CU-09 Procesar pago y factura	9
3	Diagrama de clases	10
3.1	Decisiones de diseño y mejoras	11
3.2	Enumeraciones	11
3.3	Descripción de las clases	12
3.4	Relaciones entre clases	13
4	Modelo Entidad-Relación (MySQL)	14
4.1	Tablas, llaves y tipos de datos.....	15
4.2	Correspondencia entre el diagrama de clases y las tablas.....	17
5	Diagrama de secuencia: crear pedido y enviar comanda	18
5.1	Participantes y arquitectura por capas.....	19
5.2	Flujo principal: mesa DISPONIBLE	20
5.3	Camino alternativo: mesa no disponible.....	21
5.4	Conceptos clave del diagrama	21
6	Diagrama de actividades (con carriles).....	22
6.1	Recorrido paso a paso	23

6.2	Elementos del diagrama	23
6.3	Reglas de negocio visibles	24
7	Diagrama de estados de la mesa	1
7.1	Estados	1
7.2	Transiciones	1
7.3	Análisis	2
8	Diagrama de despliegue	1
8.1	Nodos	1
8.2	Comunicaciones	1
8.3	Decisiones y consideraciones	2
9	Coherencia y trazabilidad entre diagramas	3
10	Conclusiones	4

1 Introducción

1.1 Propósito

La Fase 3 del proyecto consiste en modelar el sistema mediante UML (Unified Modeling Language) antes de programarlo. Los diagramas permiten que todo el equipo comparta una misma visión del sistema: qué debe hacer, cómo se organizan sus datos y su código, cómo fluye el trabajo entre las personas y los componentes, y en qué infraestructura va a ejecutarse.

Este documento reúne, explica y justifica cada uno de los diagramas de RestoSoft G4, un sistema de punto de venta (POS) para restaurante que gestiona mesas, pedidos, comandas de cocina, facturación y cierre de caja.

1.2 Contexto del sistema

RestoSoft G4 es una aplicación que opera dentro de la red local (LAN) del restaurante, sin depender de internet. Su flujo de negocio básico es:

- El mesero abre un pedido sobre una mesa y envía la comanda a cocina o barra.
- El cocinero o barman prepara los productos y marca la comanda como lista.
- El mesero sirve, imprime la pre-factura y el cajero cobra, liquidando impuestos y propina.
- El administrador gestiona el menú, autoriza anulaciones con su clave y consulta reportes.

Tecnológicamente se construye con Java 17 y Spring Boot (aplicación servidor, API REST y WebSocket) y MySQL 8.0 (base de datos), con una arquitectura por capas: controlador → servicio → repositorio.

1.3 Diagramas incluidos

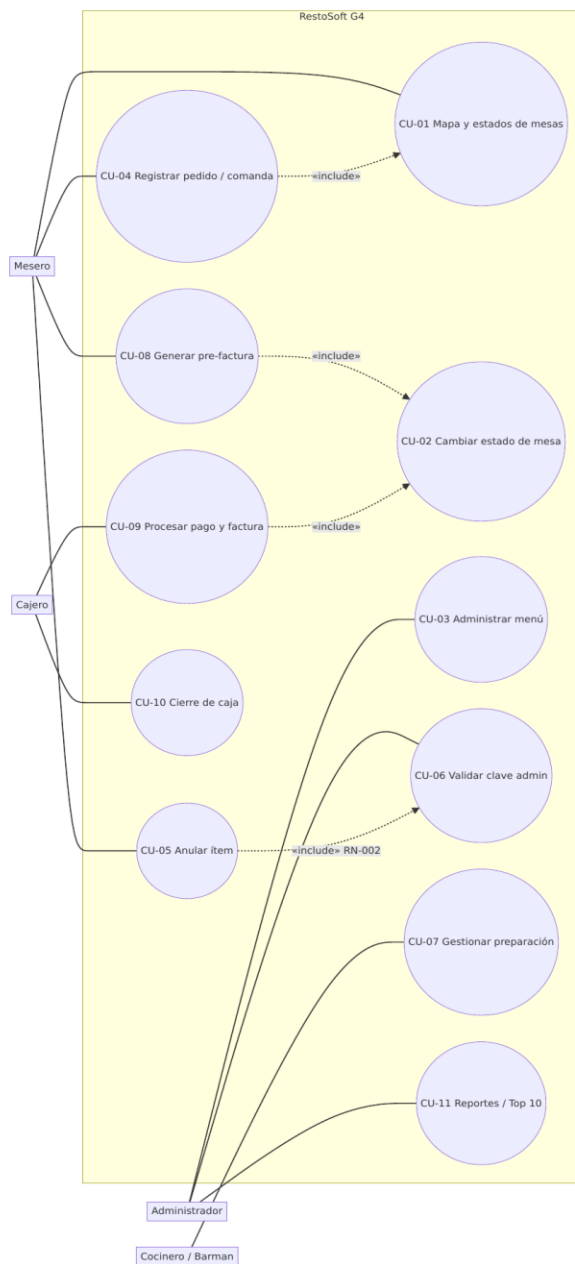
Diagrama	Requisito de la fase	Qué responde	Sección
Casos de uso	Obligatorio	¿Qué hace el sistema y quién lo usa?	2
Clases	Obligatorio	¿Cómo se estructura el código orientado a objetos?	3
Secuencia	Obligatorio	¿Cómo colaboran los objetos en el tiempo?	5
Actividades (con carriles)	Obligatorio	¿Cómo fluye el proceso entre los actores?	6
Estados (mesa)	Opcional	¿Qué ciclo de vida tiene una mesa?	7
Despliegue	Opcional	¿Dónde se ejecuta cada parte del sistema?	8
Componentes	Opcional	No se incluye como diagrama independiente; la arquitectura por capas se describe en la sección 5 y la infraestructura en la 8.	—
Entidad-Relación (MySQL)	Complementario	¿Cómo se almacenan los datos?	4

1.4 Convenciones y códigos usados

Código	Significado	Ejemplo
CU-xx	Caso de uso	CU-04 Registrar pedido / comanda
HU-xx	Historia de usuario (requerimientos ágiles)	HU-08 Abrir pedido
RF-xxx	Requisito funcional	RF-001 Apertura de pedido
RN-xxx	Regla de negocio	RN-001 Liberación automática de la mesa
PK / FK / UK	Llave primaria / foránea / restricción de unicidad	PEDIDO.mesa_id (FK)
+ / -	Visibilidad pública / privada en el diagrama de clases	-Long id ; +calcularTotales()

2 Diagrama de casos de uso

Muestra qué funciones ofrece el sistema y qué actores las utilizan. Las funciones están agrupadas dentro de la frontera del sistema («RestoSoft G4») y la dependencia entre ellas se expresa con relaciones «include».



2.1 Actores

Un actor es cualquier persona o sistema externo que interactúa con RestoSoft G4. Se identificaron cuatro, todos humanos y todos ligados a un rol de usuario (RolUsuario) del diagrama de clases.

Actor	Rol en el restaurante	Casos de uso con los que interactúa
Mesero	Atiende las mesas, toma los pedidos y sirve.	CU-01, CU-04, CU-05, CU-08
Cajero	Cobra la cuenta y cierra la caja.	CU-09, CU-10
Cocinero / Barman	Prepara los productos de cocina y de barra.	CU-07
Administrador	Configura el menú, autoriza operaciones sensibles y consulta indicadores.	CU-03, CU-06, CU-11

2.2 Catálogo de casos de uso

Código	Caso de uso	Actor principal	Descripción
CU-01	Mapa y estados de mesas	Mesero	Muestra el plano del salón con el estado de cada mesa en colores (verde, rojo, amarillo, azul).
CU-02	Cambiar estado de mesa	(incluido)	Actualiza el estado de la mesa según avanza el servicio. No lo invoca un actor directamente; lo ejecutan otros casos de uso.
CU-03	Administrar menú	Administrador	Crea, edita y desactiva categorías y productos, y define sus precios y su destino (cocina o barra).
CU-04	Registrar pedido / comanda	Mesero	Abre un pedido para una mesa, agrega productos con observaciones y envía la comanda a cocina o barra.
CU-05	Anular ítem	Mesero	Anula un producto ya pedido, registrando el motivo y la fecha. Requiere autorización del administrador.
CU-06	Validar clave admin	Administrador	Verifica la clave del administrador para autorizar operaciones restringidas.
CU-07	Gestionar preparación	Cocinero / Barman	Consulta las comandas pendientes y actualiza su estado (en preparación, listo).
CU-08	Generar pre-factura	Mesero	Imprime la cuenta para el cliente antes del pago.
CU-09	Procesar pago y factura	Cajero	Registra el pago, aplica propina, liquida impuestos y emite la factura.
CU-10	Cierre de caja	Cajero	Compara el total del sistema con el dinero contado y registra la diferencia.
CU-11	Reportes / Top 10	Administrador	Genera reportes de ventas y el ranking de los 10 productos más vendidos.

2.3 Relaciones <<include>>

La relación «include» indica que un caso de uso siempre necesita ejecutar otro para completarse.

Se usa con estereotipo explícito para evitar ambigüedades. Es la forma de reutilizar comportamiento común sin repetirlo.

Caso base	Caso incluido	Justificación
CU-04 Registrar pedido / comanda	CU-01 Mapa y estados de mesas	Para registrar un pedido el mesero primero necesita ver y elegir una mesa disponible.
CU-08 Generar pre-factura	CU-02 Cambiar estado de mesa	Al imprimir la pre-factura la mesa pasa de OCUPADA a CERRADA.
CU-09 Procesar pago y factura	CU-02 Cambiar estado de mesa	Al pagar la mesa pasa a PAGADA y luego se libera como DISPONIBLE.
CU-05 Anular ítem	CU-06 Validar clave admin (regla RN-002)	Anular un ítem es una operación sensible: la regla RN-002 exige validar la clave del administrador siempre.

2.4 Especificación de los casos más importantes

A partir del diagrama se detallan tres casos de uso representativos. Las precondiciones y los flujos se derivan de los demás diagramas del documento.

2.4.1 CU-04 Registrar pedido / comanda

Elemento	Detalle
Actor	Mesero
Precondición	El mesero está autenticado y la mesa seleccionada está en estado DISPONIBLE.
Flujo principal	1. El mesero selecciona la mesa en el mapa (CU-01). 2. El sistema abre un pedido y cambia la mesa a OCUPADA. 3. El mesero elige productos, cantidades y observaciones. 4. El mesero pulsa «Enviar comanda». 5. El sistema crea la comanda y la muestra en la pantalla de cocina en tiempo real.
Flujo alternativo	Si la mesa ya no está disponible, el sistema rechaza la operación con error 409 y notifica «La mesa ya está ocupada».
Postcondición	Existe un pedido ABIERTO con una comanda PENDIENTE; la mesa está OCUPADA (rojo).

2.4.2 CU-05 Anular ítem

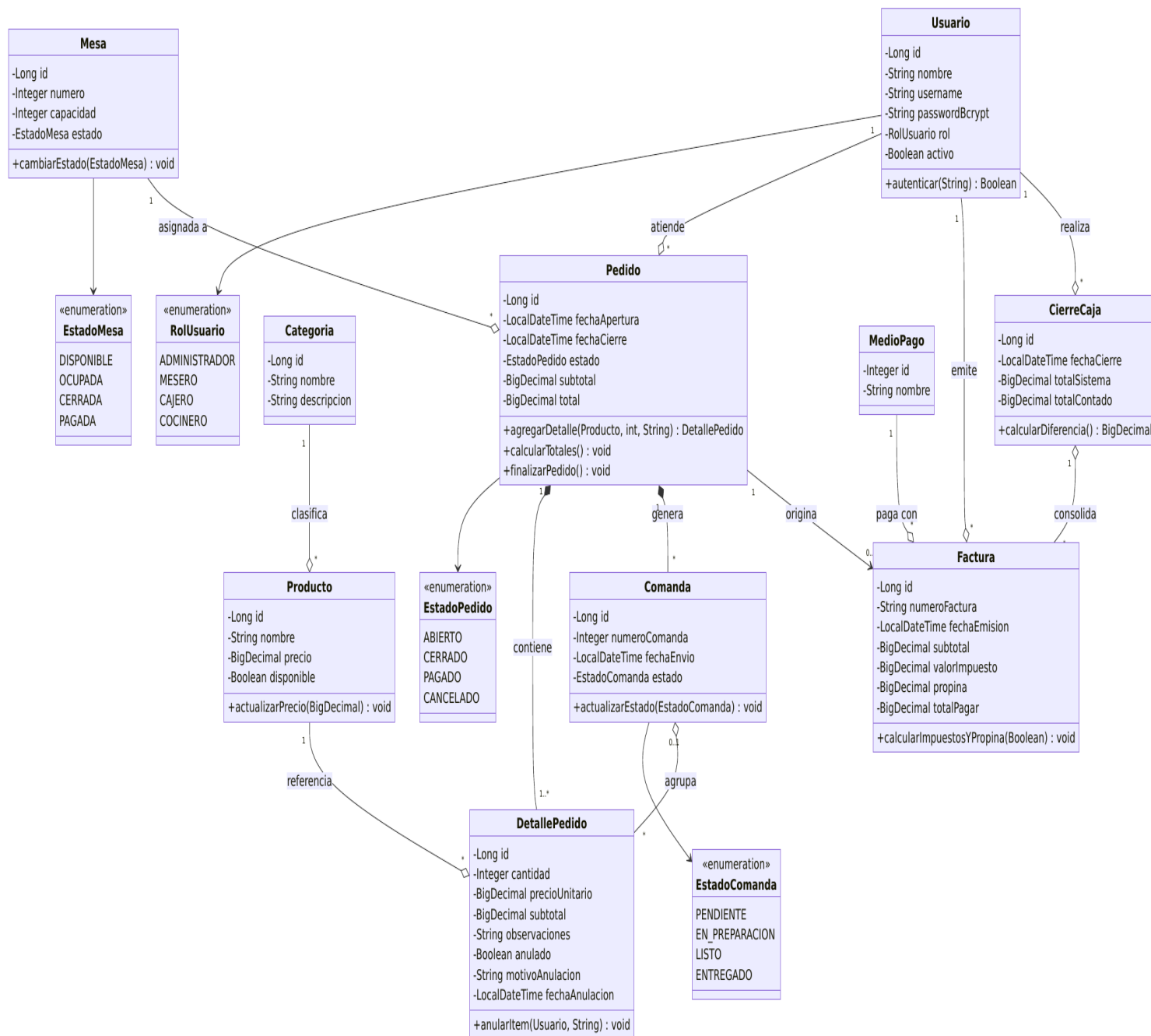
Elemento	Detalle
Actor	Mesero (con autorización del Administrador)
Precondición	Existe un pedido abierto con al menos un ítem no anulado.
Flujo principal	1. El mesero selecciona el ítem y solicita anularlo. 2. El sistema pide la clave de administrador (CU-06, regla RN-002). 3. El administrador ingresa su clave y el sistema la valida. 4. El mesero indica el motivo. 5. El sistema marca el detalle como anulado y registra quién, por qué y cuándo.
Flujo alternativo	Si la clave es incorrecta, la anulación no se realiza.
Postcondición	El ítem queda anulado con auditoría (usuario, motivo, fecha) y los totales del pedido se recalculan.

2.4.3 CU-09 Procesar pago y factura

Elemento	Detalle
Actor	Cajero
Precondición	La mesa está CERRADA (pre-factura impresa).
Flujo principal	1. El cajero recibe el pago y elige el medio de pago. 2. El cajero indica si el cliente acepta la propina del 10 %. 3. El sistema liquida IVA 19 % o Imptoconsumo 8 %. 4. El sistema genera e imprime la factura. 5. El pedido pasa a PAGADO y la mesa a PAGADA; luego se libera automáticamente (DISPONIBLE).
Postcondición	Existe una factura ligada al pedido; la mesa queda lista para nuevos clientes.

3 Diagrama de clases

Representa la estructura estática del software: las clases del dominio, sus atributos, sus métodos y cómo se relacionan.



3.1 Decisiones de diseño y mejoras

El diagrama incorpora mejoras sobre una versión anterior. Cada una responde a un problema técnico concreto:

Mejora	Por qué importa
Dinero con BigDecimal, no Double	Los tipos de punto flotante generan errores de redondeo ($0.1 + 0.2 \neq 0.3$). En un sistema de facturación esos centavos son inaceptables; BigDecimal representa montos exactos.
Anulación auditada	DetallePedido guarda anulado, motivoAnulacion y fechaAnulacion, y el método anularItem(Usuario, String) recibe quién anula. Así hay trazabilidad ante cualquier disputa.
Factura ligada a Pedido	Cada factura se origina en un pedido concreto (relación 1 a 0..1), evitando facturas «huérfanas».
DetallePedido pertenece a un Pedido y, opcionalmente, a una Comanda	Evita una «doble composición» (dos dueños con ciclo de vida exclusivo). El pedido es el dueño; la comanda solo agrupa detalles (0..1).
Nuevas clases MedioPago y CierreCaja	Permiten registrar con qué se pagó y consolidar las facturas en el cierre diario, comparando lo del sistema contra lo contado.

3.2 Enumeraciones

Una enumeración define un conjunto cerrado de valores válidos. Evita «cadenas mágicas» y errores de digitación.

Enumeración	Valores	Usada en
«enumeration» RolUsuario	ADMINISTRADOR, MESERO, CAJERO, COCINERO	Usuario.rol
«enumeration» EstadoMesa	DISPONIBLE, OCUPADA, CERRADA, PAGADA	Mesa.estado
«enumeration» EstadoPedido	ABIERTO, CERRADO, PAGADO, CANCELADO	Pedido.estado
«enumeration» EstadoComanda	PENDIENTE, EN_PREPARACION, LISTO, ENTREGADO	Comanda.estado

3.3 Descripción de las clases

Clase	Atributos	Métodos	Responsabilidad
Usuario	-id, -nombre, -username, -passwordBcrypt, -rol, -activo	+autenticar(String): Boolean	Persona que usa el sistema. La contraseña se guarda cifrada con BCrypt, nunca en texto plano.
Mesa	-id, -numero, -capacidad, -estado	+cambiarEstado(EstadoMesa): void	Mesa física del salón y su estado actual.
Categoria	-id, -nombre, -descripcion	—	Agrupar productos del menú (bebidas, platos fuertes, etc.).
Producto	-id, -nombre, -precio, -disponible	+actualizarPrecio(BigDecimal): void	Ítem que se puede vender.
Pedido	-id, -fechaApertura, -fechaCierre, -estado, -subtotal, -total	+agregarDetalle(Producto, int, String): DetallePedido +calcularTotales(): void +finalizarPedido(): void	Cuenta abierta de una mesa. Es la clase central: coordina detalles, comandas y factura.
DetallePedido	-id, -cantidad, -precioUnitario, -subtotal, -observaciones, -anulado, -motivoAnulacion, -fechaAnulacion	+anularItem(Usuario, String): void	Línea del pedido. Guarda el precio unitario del momento para que un cambio de precio posterior no altere cuentas pasadas.
Comanda	-id, -numeroComanda, -fechaEnvio, -estado	+actualizarEstado(EstadoComanda): void	Orden enviada a cocina o barra.
Factura	-id, -numeroFactura, -fechaEmision, -subtotal, -valorImpuesto, -propina, -totalPagar	+calcularImpuestosYPropina(Boolean): void	Comprobante de pago con impuesto y propina.
MedioPago	-id, -nombre	—	Forma de pago (efectivo, tarjeta, etc.).
CierreCaja	-id, -fechaCierre, -totalSistema, -totalContado	+calcularDiferencia(): BigDecimal	Cuadre de caja: detecta sobrantes o faltantes.

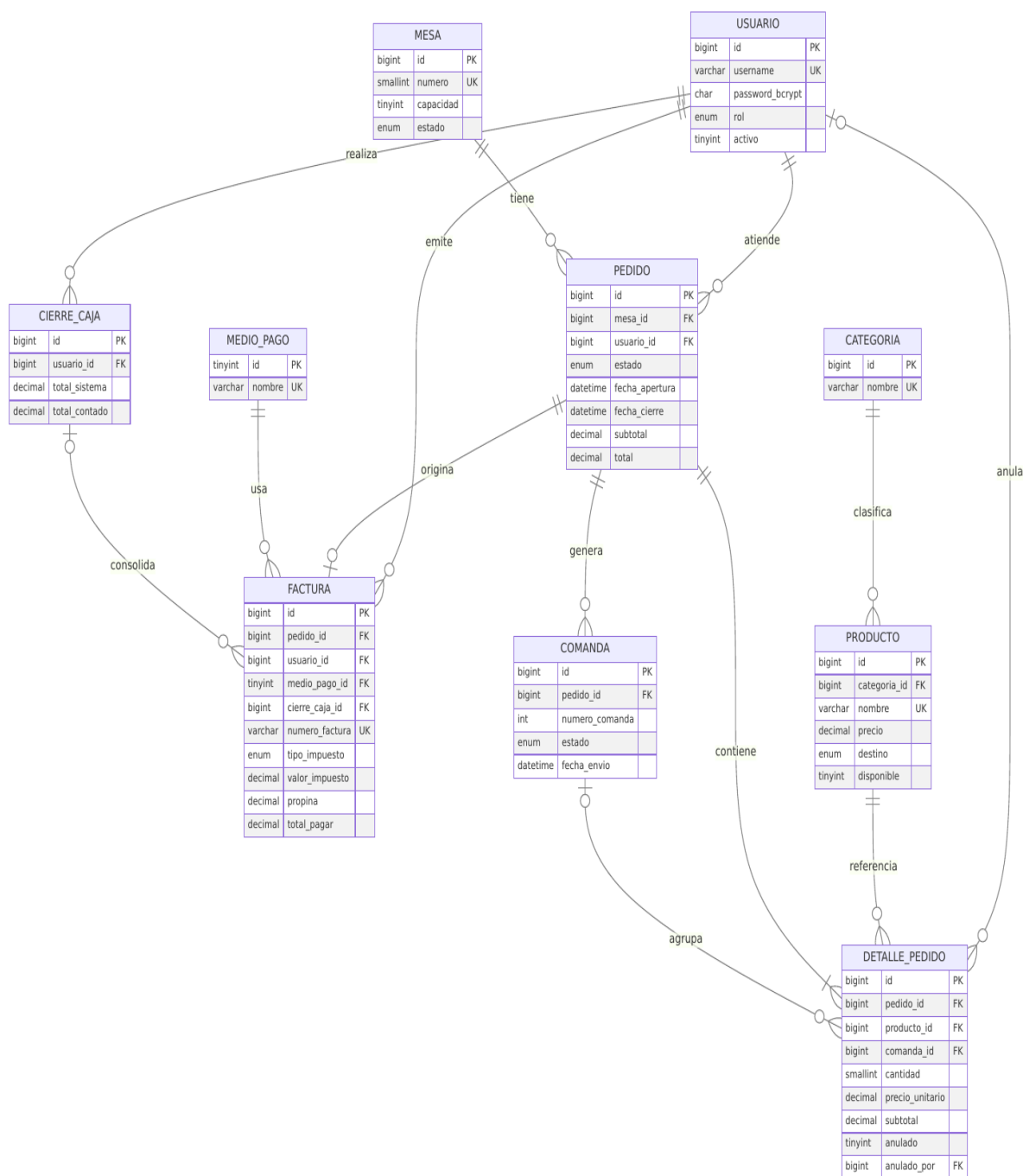
3.4 Relaciones entre clases

Las multiplicidades indican cuántos objetos participan: 1 exactamente uno, * cero o muchos, 1..* uno o más, 0..1 cero o uno.

Relación	Clases	Multiplicidad	Tipo
asignada a	Mesa — Pedido	1 — *	Agregación
atiende	Usuario — Pedido	1 — *	Agregación
contiene	Pedido — DetallePedido	1 — 1..*	Composición (rombo lleno): sin pedido no existe el detalle
genera	Pedido — Comanda	1 — *	Composición
origina	Pedido → Factura	1 — 0..1	Asociación dirigida
clasifica	Categoria — Producto	1 — *	Agregación
referencia	Producto — DetallePedido	1 — *	Agregación
agrupa	Comanda — DetallePedido	0..1 — *	Agregación (opcional)
emite	Usuario — Factura	1 — *	Agregación
paga con	MedioPago — Factura	1 — *	Agregación
consolida	CierreCaja — Factura	1 — *	Agregación
realiza	Usuario — CierreCaja	1 — *	Agregación
(enumeraciones)	Mesa, Usuario, Pedido y Comanda → su enumeración	—	Asociación dirigida

4 Modelo Entidad-Relación (MySQL)

Es la versión de base de datos del modelo de clases. Refleja las tablas del script SQL; las enumeraciones de las clases se implementan como tipos ENUM de MySQL.



4.1 Tablas, llaves y tipos de datos

Tabla	Columnas (tipo — llave)	Notas
USUARIO	id bigint PK username varchar UK password_bcrypt char rol enum activo tinyint	Contraseña cifrada con BCrypt (longitud fija, de ahí char). activo funciona como booleano.
MESA	id bigint PK numero smallint UK capacidad tinyint estado enum	El número de mesa es único.
CATEGORIA	id bigint PK nombre varchar UK	No se repiten nombres de categoría.
PRODUCTO	id bigint PK categoria_id bigint FK nombre varchar UK precio decimal destino enum disponible tinyint	destino define si la comanda del producto va a cocina o a barra.
PEDIDO	id bigint PK mesa_id bigint FK usuario_id bigint FK estado enum fecha_apertura datetime fecha_cierre datetime subtotal decimal total decimal	Une una mesa con el mesero que la atiende.
COMANDA	id bigint PK pedido_id bigint FK numero_comanda int estado enum fecha_envio datetime	Un pedido puede tener varias comandas (cada vez que se pide más).

Tabla	Columnas (tipo — llave)	Notas
DETALLE_PEDIDO	id bigint PK pedido_id bigint FK producto_id bigint FK comanda_id bigint FK (nulo) cantidad smallint precio_unitario decimal subtotal decimal anulado tinyint anulado_por bigint FK	comanda_id es opcional (aún no enviado). anulado_por apunta al usuario que anuló: es la auditoría.
MEDIO_PAGO	id tinyint PK nombre varchar UK	Tabla de catálogo pequeña.
CIERRE_CAJA	id bigint PK usuario_id bigint FK total_sistema decimal total_contado decimal	Registra el cuadro de caja.
FACTURA	id bigint PK pedido_id bigint FK usuario_id bigint FK medio_pago_id tinyint FK cierre_caja_id bigint FK numero_factura varchar UK tipo_impuesto enum valor_impuesto decimal propina decimal total_pagar decimal	numero_factura único. tipo_impuesto distingue IVA de Imposconsumo.

4.2 Correspondencia entre el diagrama de clases y las tablas

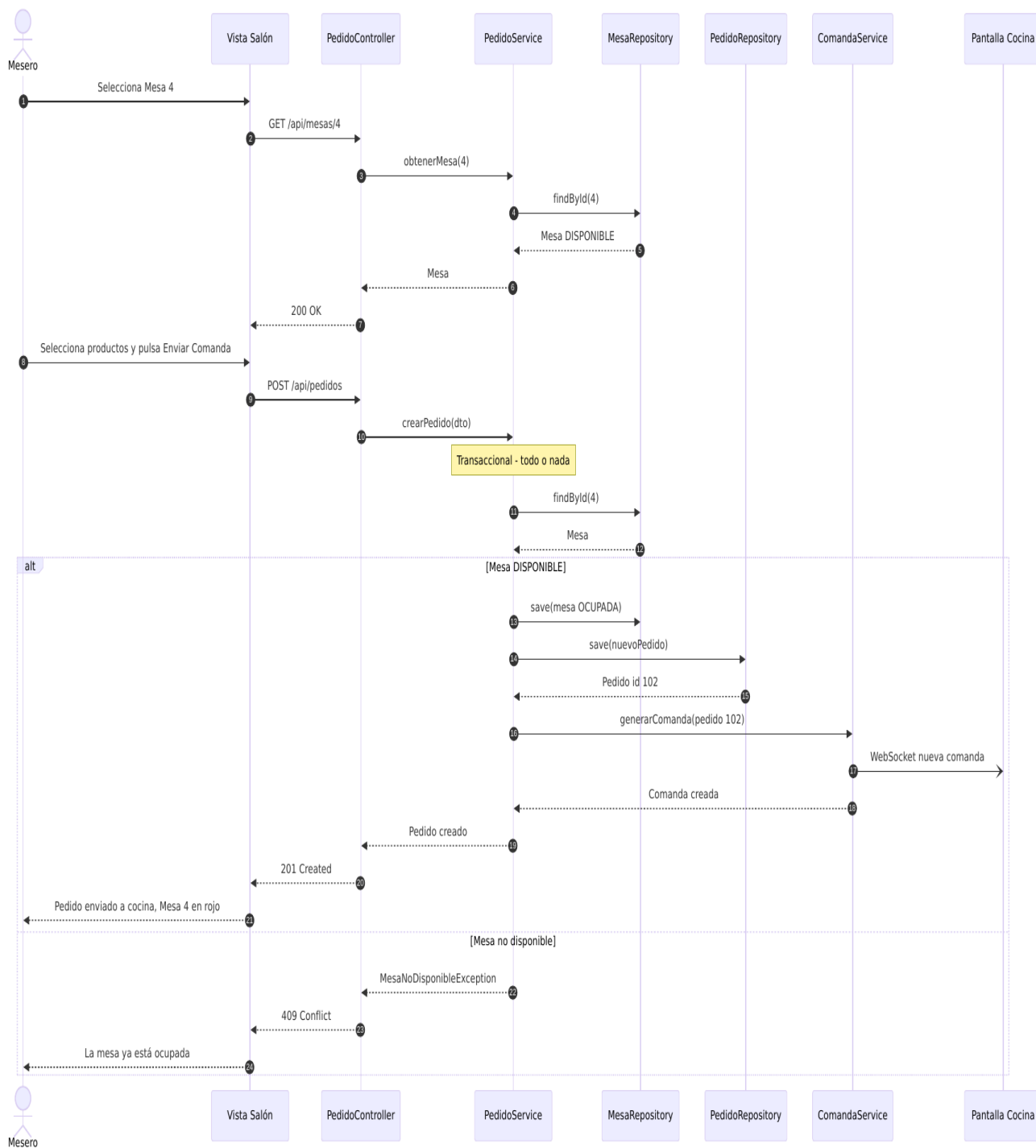
Al comparar ambos diagramas se observan diferencias menores. Probablemente el dibujo de la tabla está resumido y el script SQL sí las contiene, pero conviene verificarlas contra el script antes de entregar, para que los dos modelos coincidan:

Elemento	En el diagrama de clases	En el diagrama ER
Usuario	Tiene nombre	No aparece nombre
Categoria	Tiene descripcion	No aparece descripcion
Producto	No aparece destino	Tiene destino (enum)
DetallePedido	observaciones, motivoAnulacion, fechaAnulacion	No aparecen esas columnas
Factura	fechaEmision, subtotal	No aparecen; sí tipo_impuesto
CierreCaja	Tiene fechaCierre	No aparece fecha_cierre

5 Diagrama de secuencia: crear pedido y enviar comanda

Muestra el orden en que los participantes se envían mensajes durante un escenario concreto.

Aquí se modela el momento más crítico del servicio: abrir el pedido y notificar a cocina.



5.1 Participantes y arquitectura por capas

Los participantes siguen la arquitectura típica de Spring Boot, donde cada capa tiene una única responsabilidad:

Participante	Capa	Responsabilidad
Mesero	Actor	Persona que inicia la interacción.
Vista Salón	Presentación	Pantalla de la tablet que muestra las mesas y permite elegir productos.
PedidoController	Controlador (REST)	Recibe las peticiones HTTP, las convierte en llamadas al servicio y devuelve el código de respuesta.
PedidoService	Servicio (lógica de negocio)	Valida la mesa, abre el pedido y coordina la creación de la comanda dentro de una transacción.
MesaRepository	Repositorio (acceso a datos)	Consulta y guarda mesas.
PedidoRepository	Repositorio (acceso a datos)	Guarda pedidos.
ComandaService	Servicio	Crea la comanda y avisa a la cocina.
Pantalla Cocina	Presentación (KDS)	Recibe la comanda en tiempo real por WebSocket.

5.2 Flujo principal: mesa DISPONIBLE

Paso	Emisor → Receptor	Mensaje
1	Mesero → Vista Salón	Selecciona Mesa 4
2	Vista → Controller	GET /api/mesas/4
3	Controller → Service	obtenerMesa(4)
4	Service → MesaRepository	findById(4)
5–6	MesaRepository → Service → Controller	Mesa DISPONIBLE
7	Controller → Vista	200 OK
8	Mesero → Vista	Selecciona productos y pulsa «Enviar Comanda»
9	Vista → Controller	POST /api/pedidos
10	Controller → Service	crearPedido(dto)
11–12	Service ↔ MesaRepository	findById(4) → Mesa
13	Service → MesaRepository	save(mesa OCUPADA)
14–15	Service ↔ PedidoRepository	save(nuevoPedido) → Pedido id 102
16	Service → ComandaService	generarComanda(pedido 102)
17	ComandaService → Pantalla Cocina	WebSocket: nueva comanda
18	ComandaService → Service	Comanda creada
19–20	Service → Controller → Vista	Pedido creado — 201 Created
21	Vista → Mesero	Pedido enviado a cocina, Mesa 4 en rojo

5.3 Camino alternativo: mesa no disponible

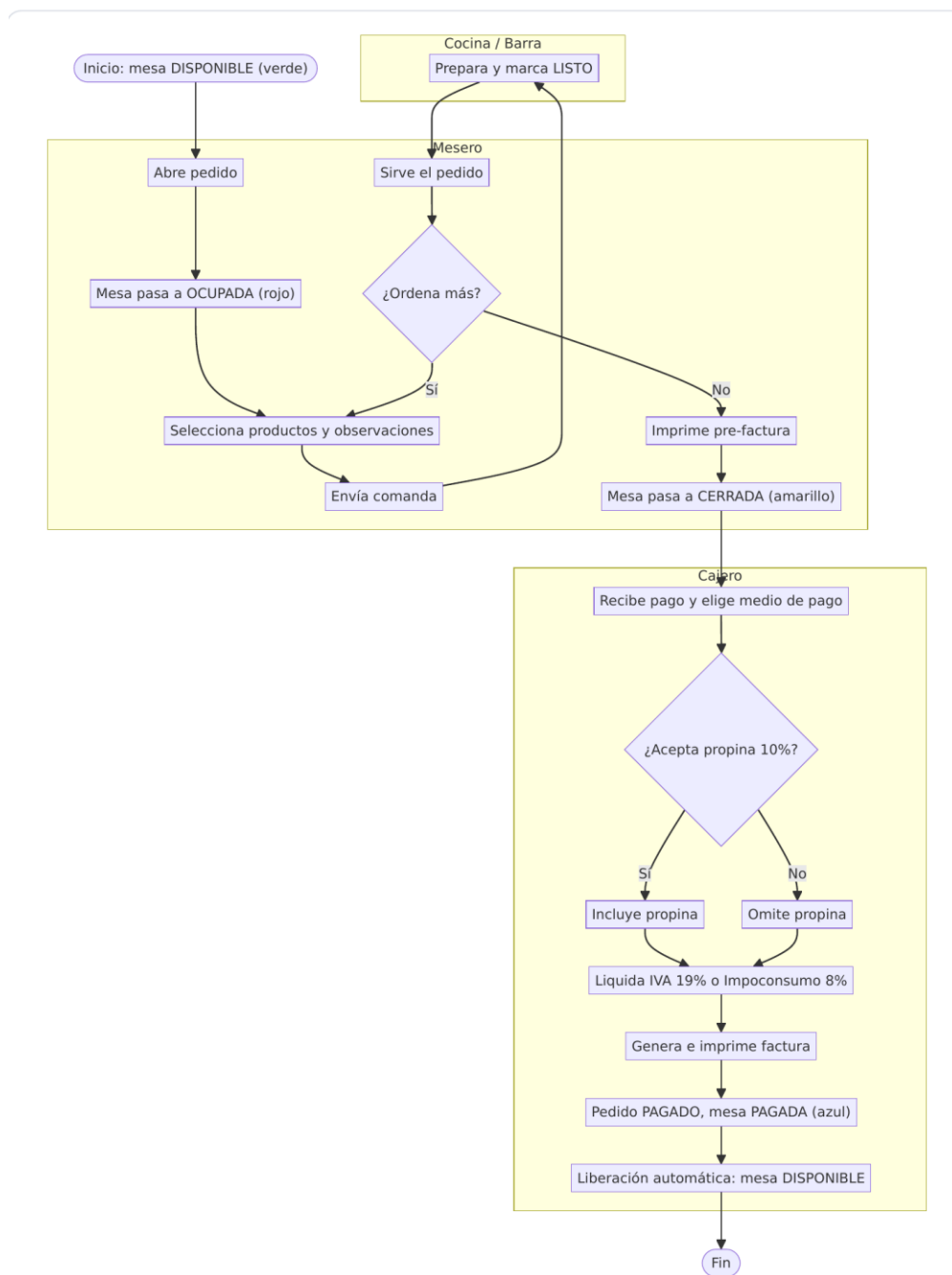
Si al crear el pedido la mesa ya no está DISPONIBLE (por ejemplo, otro mesero la tomó un instante antes), el servicio lanza `MesaNoDisponibleException`. El controlador la traduce a 409 Conflict y la vista muestra «La mesa ya está ocupada». Esta validación protege contra condiciones de carrera entre varias tablets.

5.4 Conceptos clave del diagrama

- Bloque transaccional («todo o nada»): los pasos de guardar la mesa, guardar el pedido y generar la comanda se ejecutan como una sola unidad. Si algo falla a mitad, se revierte todo y no queda una mesa ocupada sin pedido.
- Fragmento alt: representa la decisión entre el camino normal y el de error.
- Códigos HTTP: 200 OK (consulta correcta), 201 Created (recurso creado), 409 Conflict (conflicto con el estado actual).
- WebSocket: canal bidireccional permanente; permite a la cocina ver la comanda al instante sin tener que consultar al servidor repetidamente

6 Diagrama de actividades (con carriles)

Describe el flujo de trabajo completo del servicio, separando las responsabilidades por actor en carriles (swimlanes): Mesero, Cocina/Barra y Cajero.



6.1 Recorrido paso a paso

Carril	Actividad	Efecto sobre la mesa
—	Inicio: la mesa está lista para clientes.	DISPONIBLE (verde)
Mesero	Abre pedido.	Pasa a OCUPADA (rojo)
Mesero	Selecciona productos y observaciones.	—
Mesero	Envía comanda.	—
Cocina / Barra	Prepara y marca LISTO.	—
Mesero	Sirve el pedido.	—
Mesero	Decisión: ¿ordena más? Sí → vuelve a seleccionar productos. No → continúa.	—
Mesero	Imprime pre-factura.	Pasa a CERRADA (amarillo)
Cajero	Recibe pago y elige medio de pago.	—
Cajero	Decisión: ¿acepta propina 10 %? Sí → incluye propina. No → omite propina.	—
Cajero	Liquida IVA 19 % o Impoconsumo 8 %.	—
Cajero	Genera e imprime factura.	—
Sistema	Pedido PAGADO, mesa PAGADA.	PAGADA (azul)
Sistema	Liberación automática. Fin.	Vuelve a DISPONIBLE

6.2 Elementos del diagrama

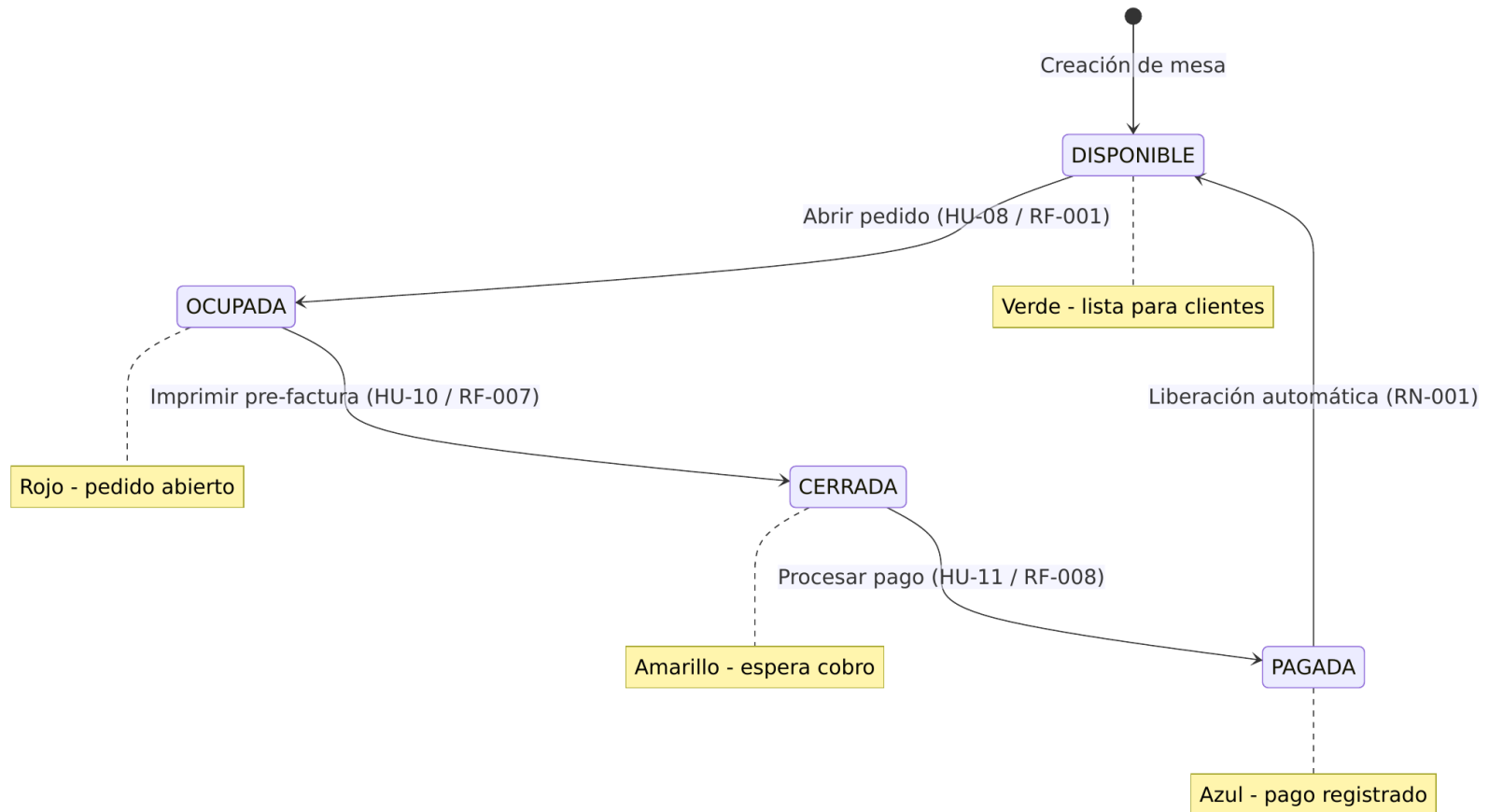
- Carriles (swimlanes): columnas que indican quién es responsable de cada actividad.
Facilitan ver las entregas de trabajo entre actores.
- Nodos de decisión (rombos): «¿Ordena más?» crea un ciclo, porque una mesa puede pedir varias rondas; «¿Acepta propina 10 %?» bifurca el cálculo de la cuenta.
- Nodo inicial y final: marcan el principio y el fin del proceso.

6.3 Reglas de negocio visibles

- La propina sugerida es del 10 % y es opcional: el cliente decide.
- Se liquida IVA del 19 % o Impoconsumo del 8 %, según el tipo de impuesto aplicable al establecimiento (campo tipo_impuesto de FACTURA).
- La mesa se libera automáticamente al pagar (regla RN-001), sin que nadie tenga que hacerlo manualmente.
- Coherencia con el diagrama de estados (sección 7): el cierre PAGADA → DISPONIBLE coincide en ambos diagramas.

7 Diagrama de estados de la mesa

Modela el ciclo de vida de una mesa: en qué estados puede estar y qué evento provoca cada cambio.



7.1 Estados

Estado	Color	Significado
DISPONIBLE	Verde	Lista para recibir clientes.
OCUPADA	Rojo	Tiene un pedido abierto.
CERRADA	Amarillo	Se imprimió la pre-factura; espera el cobro.
PAGADA	Azul	El pago quedó registrado.

7.2 Transiciones

Desde	Hacia	Evento disparador	Trazabilidad
● (inicio)	DISPONIBLE	Creación de mesa	—
DISPONIBLE	OCUPADA	Abrir pedido	HU-08 / RF-001
OCUPADA	CERRADA	Imprimir pre-factura	HU-10 / RF-007
CERRADA	PAGADA	Procesar pago	HU-11 / RF-008
PAGADA	DISPONIBLE	Liberación automática	RN-001

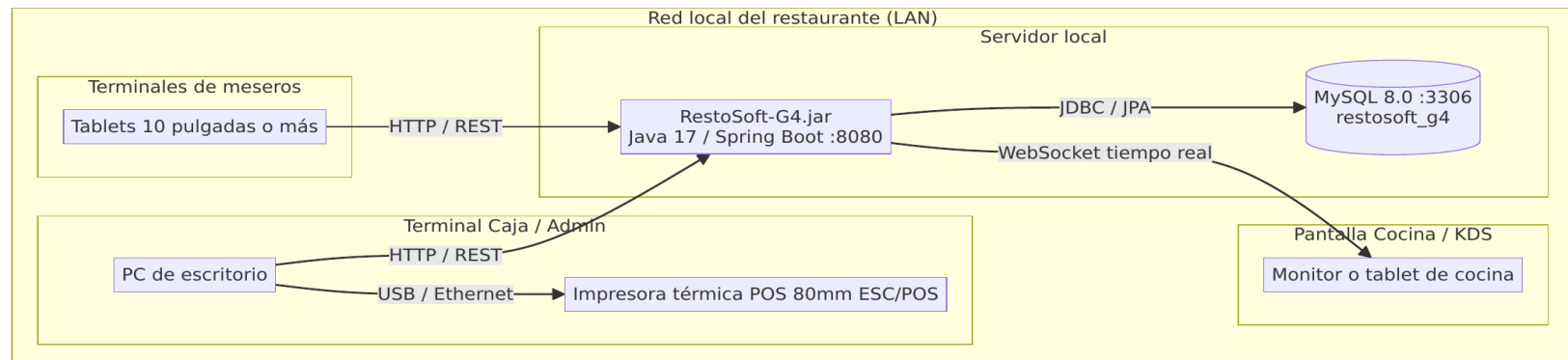
7.3 Análisis

- El ciclo es lineal y cerrado: la mesa solo avanza en un sentido y vuelve al inicio al final, lo que evita estados inconsistentes.
- Cada transición está ligada a un requisito (HU, RF o RN), de modo que se puede verificar que cada comportamiento tiene respaldo en el SRS.
- Los colores se documentan como notas en el diagrama y son los que muestra el mapa de mesas (CU-01).
- La única transición automática es PAGADA → DISPONIBLE; las demás las dispara una acción del usuario.

En el código, estas reglas se implementan en `Mesa.cambiarEstado(EstadoMesa)` y se invocan desde los casos de uso CU-02, CU-04, CU-08 y CU-09.

8 Diagrama de despliegue

Muestra la infraestructura física y dónde se ejecuta cada parte del software, con los puertos y protocolos de comunicación. Todo opera dentro de la LAN del restaurante.



8.1 Nodos

Nodo	Hardware	Software / artefactos
Servidor local	Equipo dentro del restaurante	RestoSoft-G4.jar (Java 17 / Spring Boot) en el puerto :8080 MySQL 8.0 en el puerto :3306, base de datos restosoft_g4
Terminales de meseros	Tablets de 10 pulgadas o más	Interfaz del salón (navegador o cliente web)
Terminal Caja / Admin	PC de escritorio	Interfaz de caja y administración
Pantalla Cocina / KDS	Monitor o tablet de cocina	Pantalla de comandas (Kitchen Display System)
Impresora térmica	POS 80 mm, protocolo ESC/POS	Impresión de pre-facturas y facturas

8.2 Comunicaciones

Origen	Destino	Protocolo	Uso
Terminales de meseros	Servidor (Spring Boot :8080)	HTTP / REST	Mapa de mesas, pedidos, comandas.
Terminal Caja / Admin	Servidor (Spring Boot :8080)	HTTP / REST	Pagos, facturas, cierre de caja, menú y reportes.
Servidor	Pantalla Cocina / KDS	WebSocket (tiempo real)	Envío inmediato de nuevas comandas.
Aplicación Spring Boot	MySQL (:3306)	JDBC / JPA	Lectura y escritura de datos.
Terminal Caja	Impresora térmica	USB / Ethernet	Impresión de documentos.

8.3 Decisiones y consideraciones

- Operación en LAN: el servicio no se interrumpe si se cae internet y la latencia es mínima, algo esencial en la hora pico del restaurante.
- Servidor único: aplicación y base de datos conviven en el mismo equipo, lo que simplifica la instalación y el mantenimiento.
- Consideración de riesgo: al ser un único servidor, es el punto único de falla. Conviene planificar copias de seguridad periódicas de restosoft_g4.
- Puertos explícitos: documentar :8080 y :3306 facilita configurar el firewall y los clientes.

9 Coherencia y trazabilidad entre diagramas

Un buen modelado exige que los diagramas no se contradigan. La siguiente tabla muestra cómo un mismo concepto aparece en cada diagrama:

Concepto	Casos de uso	Clases / ER	Secuencia	Actividades / Estados
Abrir pedido	CU-04	Pedido, Mesa	POST /api/pedidos; mesa a OCUPADA	Mesero abre pedido; DISPONIBLE → OCUPADA
Enviar comanda	CU-04, CU-07	Comanda	generarComanda + WebSocket	Mesero envía; cocina prepara
Anular ítem	CU-05, CU-06	DetallePedido.anulado, anulado_por	—	—
Pre-factura	CU-08	Pedido.calcularTotales()	—	OCUPADA → CERRADA
Pago y factura	CU-09	Factura, MedioPago	—	Cajero liquida; CERRADA → PAGADA
Liberar mesa	CU-02	Mesa.cambiarEstado()	—	PAGADA → DISPONIBLE (RN-001)
Cierre de caja	CU-10	CierreCaja, Factura.cierre_caja_id	—	—
Cocina en tiempo real	CU-07	Comanda.estado	WebSocket	Cocina / Barra; despliegue: KDS

10 Conclusiones

- El modelo cubre el ciclo completo de operación de un restaurante: desde que se ocupa una mesa hasta que se factura, se cobra y se cuadra la caja.
- Se entregan los cuatro diagramas obligatorios (casos de uso, clases, secuencia, actividades) y se complementan con estados, despliegue y el modelo Entidad-Relación.
- El diseño incorpora buenas prácticas: dinero con BigDecimal, contraseñas con BCrypt, anulaciones auditadas, transacciones «todo o nada», códigos HTTP correctos y liberación automática de mesas.
- Los diagramas están alineados entre sí (por ejemplo, actividades y estados coinciden en el cierre del ciclo de la mesa) y trazados a los requisitos HU, RF y RN.
- Como trabajo siguiente se recomienda verificar las diferencias señaladas en la sección 4.3 contra el script SQL y, si el docente lo solicita, agregar un diagrama de componentes.